The problem: When a node receives a block via `node_message` and stores it, it stops there. In a network of $3+$ nodes where node C sends to node A, node B never hears about it.

What to build: After `receiveBlock` succeeds in `node_message`, rebroadcast the block to all other connected peers except the one it came from:

```
if action == Action.registeration.value:
  try:
    block = Block.model_validate(payload)
    self.blockManager.receiveBlock(block)

    # Gossip to all other peers
    self.send_to_nodes({
      "action": Action.registeration.value,
      "data": payload
    }, execlude = [node]) # don't send it back to the sender.

    except ...
```

I also need a `seenBlock` set on `Peer` to prevent infinite rebroadcast loop:

```
# In Peer.__init__
self.seenBlocks: list = []

# In node_message
chid: str = payload.get("data")
if (chid in self.seenBlocks):
  return # Already processed, don't boradcast.

self.seenBlocks.insert(chid)
```